Banking System

Sprint 1: System Design

Table of Contents

Table of Contents **2**
System Interaction with the Environment **2**
Dependencies 2
Assumptions 2
CRC Cards **2**
Frontend **6**
Overview 6
Components 6
Dashboard 6
Deposit 6
E-Transfer 6
Login 6

System Interaction with the Environment

Dependencies

1. NodeJS is used to handle server-side programming and dependency importing.
2. Angular framework is used to generate the frontend view for customers.
3. Java is used for backend logic to manage the database.
4. MySQL database is used to model and store data.

Assumptions

1. The system will be deployed and accessed over a modern web browser.
2. An internet connection is necessary for functionality of the system.

CRC Cards

Class:AccountController	
Parent Class: N/A Subclasses: N/A	
Responsibilities: • Handle CRUD operations	Collaborators: N/A

○ Post ○ Get ○ Put ○ Delete	

Class:CustomerInfoController	
Parent Class: N/A Subclasses:N/A	
Responsibilities: ● Handle CRUD operations ○ Post ○ Get ○ Put ○ Delete	Collaborators: N/A

Class:AccountDAO	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Provide an abstract for expected account operations	Collaborators: ● AccountService

Class:CustomerInfoDAO	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Provide an abstract for expected customer operations	Collaborators: ● CustomerInfoService

Class:LocalAccountDataAccessService
Parent Class: N/A

Subclasses: N/A	
Responsibilities: ● Provide an implementation for account operations	Collaborators: AccountDAO

Class:LocalCustomerDataAccessService	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Provide an implementation for customer operations	Collaborators: CustomerDAO

Class:Account	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Models the attributes and methods of accounts ○ ID, balance ● Getters and setters	Collaborators: ● AccountOwners ● CustomerInfo ● Transaction ● AccountService

Class:AccountOwners	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Assigns unique owners to their respective account	Collaborators: ● Account

Class:CustomerInfo	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Models the attributes and	Collaborators: ● Account

methods of accounts ○ ID, email, password, mobile number, address ● Getters and Setters	● SavedPayees ● AccountService

Class:Transaction	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Models the attributes of a transaction ○ ID, deposit/withdraw, amount, date ● Getters and Setters	Collaborators: ● CustomerInfo ● Account

Class:AccountService	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Applies the operations of Accounts while associating other classes ○ Account creation, Balance change ● Getters and Setters	Collaborators: ● AccountDAO ● Account ● CustomerInfo ● Transaction

Class:CustomerInfoService	
Parent Class: N/A Subclasses: N/A	
Responsibilities: ● Applies the operations of CustomerInfo while associating other classes ○ Add new info ● Getters and Setters	Collaborators: ● CustomerDAO

Frontend

Overview

The frontend can be broken down into the various menus the user can access. With exception to the initial login page, every component can be interacted with in any order.

Components

Dashboard

- Fetches an account's balance and recent transactions

Deposit

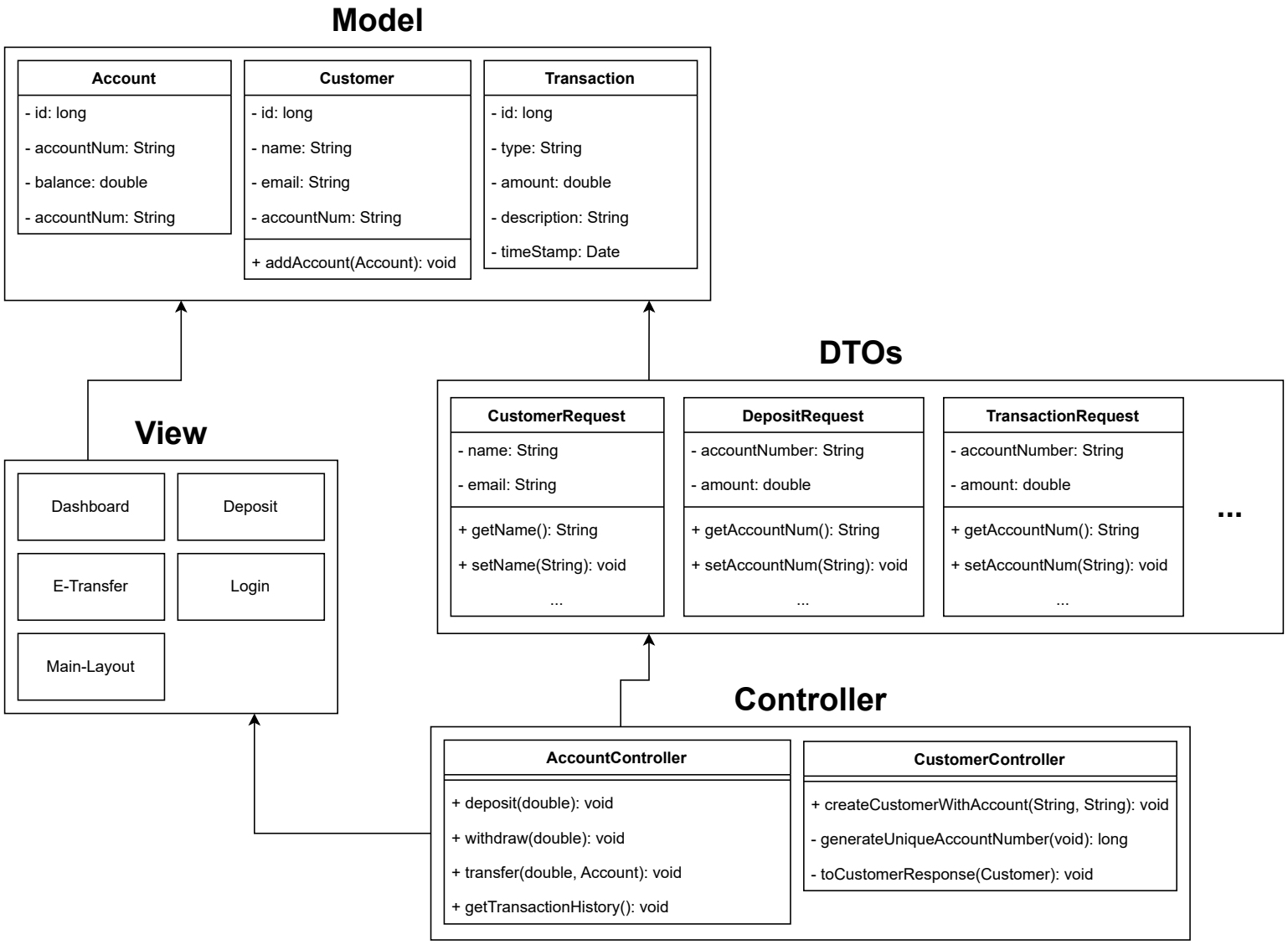
- Prompts the user to input an amount of money to the owner's account

E-Transfer

- Prompts the user to input an amount of money to send to another user's account

Login

- Requested the user's email and password
- Allows access if the credentials are found in the database


Model
Account
- id: long
- accountNum: String
- balance: double
- accountNum: String
Customer
- id: long
- name: String
- email: String
- accountNum: String
+ addAccount(Account): void
Transaction
- id: long
- type: String
- amount: double
- description: String
- timeStamp: Date
DTOs
CustomerRequest
- name: String
- email: String
+ getName(): String
+ setName(String): void
...
DepositRequest
- accountNumber: String
- amount: double
+ getAccountNum(): String
+ setAccountNum(String): void
...
TransactionRequest
- accountNumber: String
- amount: double
+ getAccountNum(): String
+ setAccountNum(String): void
...
...
View
Dashboard
Deposit
E-Transfer
Login
Main-Layout
Controller
AccountController
+ deposit(double): void
+ withdraw(double): void
+ transfer(double, Account): void
+ getTransactionHistory(): void
CustomerController
+ createCustomerWithAccount(String, String): void
- generateUniqueAccountNumber(void): long
- toCustomerResponse(Customer): void